



AI & LLM Foundation

AICB-P1 · Ngày 1 · Nền tảng

Huỳnh Thành Trung

VinUniversity · Phase 1 · Tuần 1 · 02/04/2026

HÃY Suy Nghĩ...

*“Bạn đang dùng AI mỗi ngày —
nhưng thực sự bên trong nó làm gì?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Bức tranh AI 2026
2. LLM — Trái tim của AI hiện đại
3. Token Economy
4. Gọi API lần đầu
5. Vibe Coding
6. Thực hành

Bức tranh AI 2026

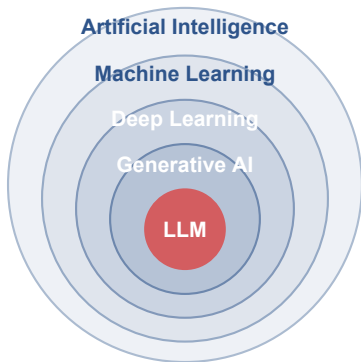
Từ Machine Learning đến Agentic AI

01

Sau buổi học này, bạn sẽ:

1. Hiểu cách LLM hoạt động (Transformer, token, next-token prediction)
2. Ước tính chi phí API call dựa trên token economy
3. Sử dụng LLM từ third-party (OpenAI, Anthropic) hoặc self-host open model
4. Nắm vững Vibe Coding mindset và sử dụng AI đúng cách, không lệ thuộc
5. Xây dựng chatbot đơn giản có streaming response

Python 3.10+, VS Code/Cursor, API key (OpenAI)



- **AI:** Máy thực hiện tác vụ “thông minh”
- **ML:** Học từ dữ liệu, không cần lập trình tường minh
- **DL:** Neural networks nhiều tầng
- **Generative AI:** Nhánh AI tiên tiến có khả năng sáng tạo ra nội dung (văn bản, ảnh, video) giống như con người
- **LLM:** Foundation Model chuyên ngôn ngữ — nền tảng của GenAI và Agentic AI

Khóa học này tập trung vào LLM → xây dựng Agentic AI

Discriminative AI

Chức năng: Phân loại, dự đoán

Ví dụ:

- Spam filter
- Image classifier
- Fraud detection

Input → Label

Generative AI

Chức năng: Sinh nội dung mới

Ví dụ:

- ChatGPT, Claude
- DALL-E, Midjourney
- GitHub Copilot

Prompt → Content

Agentic AI

Chức năng: Tự lập kế hoạch & hành động

Ví dụ:

- AI coding agents
- Auto customer support
- Research agents

Goal → Plan → Action

LLM là **engine chung** cho cả Generative AI lẫn Agentic AI

Hành trình khóa học: LLM Foundation → Agent → Multi-Agent → Deploy → Evaluate



78%

Doanh nghiệp
dùng AI

\$15.7T

GDP toàn cầu
từ AI (2030)

3.7x

ROI trung bình
trên mỗi \$1 đầu tư

AI không còn chỉ là “trả lời hay” nữa. Từ 2024 trở đi, doanh nghiệp quan tâm nhiều hơn đến AI biết **hành động, kết nối công cụ và tạo ra ROI**.

Level 0 — Core reasoning engine

LLM suy luận dựa trên kiến thức nội tại của chính nó, không sử dụng công cụ bên ngoài.

Level 1 — Connected Solver

LLM trở thành agent, có khả năng kết nối với công cụ bên ngoài, truy xuất dữ liệu, tìm kiếm, gọi API.

Level 2 — Strategic Problem-Solver

LLM agent lập kế hoạch nhiều bước. Sử dụng nhiều công cụ và chuỗi suy luận để xử lý bài toán phức tạp.

Level 3 — Collaborative AI Agents

Nhiều agent LLM chuyên biệt phối hợp làm việc với nhau để giải quyết vấn đề phức tạp.

Prompt tĩnh chỉ giải quyết 1 câu hỏi

- Prompt → Response (1 bước)
- Không truy cập dữ liệu mới
- Không hành động được

AI Agent giải quyết mục tiêu hoàn chỉnh

- Goal → Plan → Action
- Kết nối API, database, tools
- Xử lý workflow nhiều bước
- Tạo giá trị thực tế (ROI)

- **Goal** — nhận mục tiêu thay vì prompt đơn lẻ
- **Reasoning** — phân tích và lập kế hoạch nhiều bước
- **Tools** — search, API, database, code
- **Memory** — lưu trạng thái và lịch sử
- **Action** — thực thi hành động trong hệ thống

Agent = Goal + Reasoning + Tools + Memory + Action

- **Generalist AI:** Agent chuyển từ chuyên biệt → AI tổng quát xử lý mục tiêu phức tạp, dài hạn
- **Deep Personalization:** AI cá nhân hóa sâu, chủ động đề xuất và khám phá mục tiêu người dùng
- **Embodied AI:** AI tích hợp vào robot, IoT và hệ thống thế giới vật lý
- **Agent-driven Economy:** AI agents tự vận hành, tham gia kinh tế và tự động hóa lao động
- **Adaptive Multi-Agent Systems:** Hệ multi-agent tự đánh giá, tạo/nhân bản/loại bỏ agent để tối ưu nhiệm vụ

LLM — Trái tim của AI hiện đại

Transformer, Token, và cách LLM “suy nghĩ”

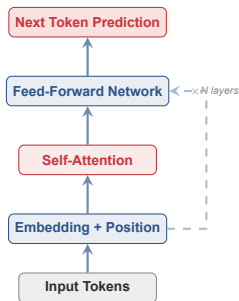
02

Large Language Model (LLM)

Mô hình ngôn ngữ lớn dựa trên kiến trúc **Transformer**, được huấn luyện trên lượng dữ liệu văn bản khổng lồ (hàng nghìn tỷ token). LLM có khả năng sinh văn bản, trả lời câu hỏi, viết code, và thực hiện reasoning phức tạp.

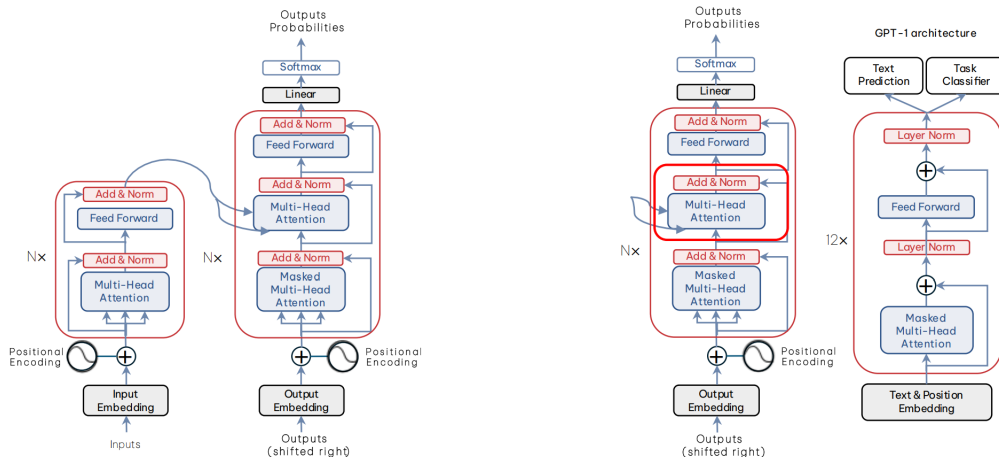
Đặc điểm chính:

- Decoder-only Transformer architecture
- Self-supervised pre-training + RLHF fine-tuning
- Next-token prediction — dự đoán từ tiếp theo
- Emergent capabilities xuất hiện khi scale lên



- **Self-Attention:** Mỗi token “nhìn” tất cả token khác trong context
- **Multi-Head:** Nhiều “góc nhìn” song song
- **Feed-Forward:** Xử lý phi tuyến từng vị trí
- **Residual connections:** Gradient chảy dễ dàng

Hai kiến trúc chính: Encoder-Decoder (BERT, T5) hiểu ngữ cảnh 2 chiều để phân loại, dịch thuật. **Decoder-only** (GPT, Claude, Gemini) đọc trái→phải để dự đoán token tiếp và sinh văn bản. *Ngày nay Decoder-only thắng thế nhờ scale tốt hơn.*



Hai kiến trúc chính: Encoder-Decoder (BERT, T5) (hình bên trái) hiểu ngữ cảnh 2 chiều để phân loại, dịch thuật. Decoder-only (GPT, Claude, Gemini) (hình bên phải) đọc trái→phải để dự đoán token tiếp và sinh văn bản. *Ngày nay Decoder-only thắng thế nhờ scale tốt hơn.*

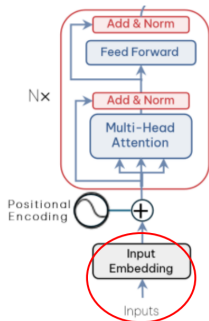
Transformer xử lý dữ liệu song song, không phải tuần tự.



[Con]	[mèo]	[ngồi]	[trên]	[bàn]
-------	-------	--------	--------	-------



[Bàn]	[ngồi]	[trên]	[con]	[mèo]
-------	--------	--------	-------	-------



Input embedding cho mô hình biết “đây là từ gì?”, nhưng nó không hề biết “từ này đứng ở đâu?”. Nếu model chỉ nhìn vào danh sách từ, **nó sẽ đánh giá 2 câu này là một.**

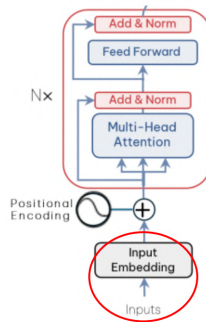
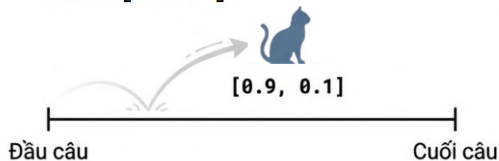
Embedding chỉ mang danh tính, không mang tọa độ.

$$E_{token} = LookupTable(token)$$

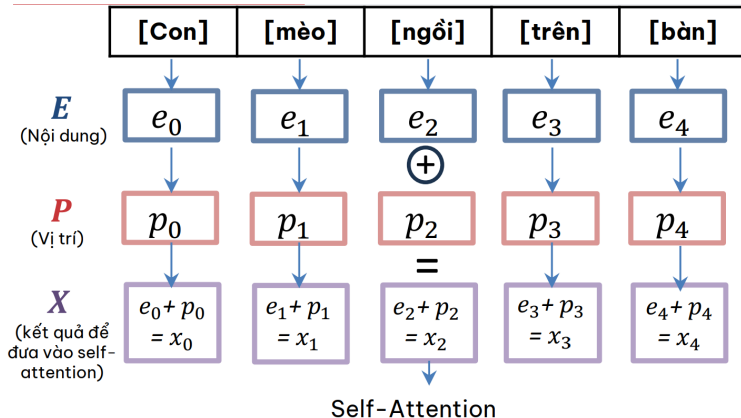
Với E là vector đại diện cho ý nghĩa từ (được học từ từ từ điển).

Token	Embedding
Con	[0.2, 0.8]
mèo	[0.9, 0.1]
ngồi	[0.7, 0.6]
trên	[0.3, 0.5]
bàn	[0.8, 0.4]

Từ “mèo” ở vị trí số 1 hay số 100 thì vector của nó vĩnh viễn là [0.9, 0.1].



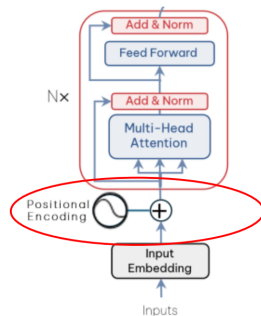
Làm thế nào Transformer hiểu được thứ tự khi nó đọc toàn bộ câu cùng một lúc?



Positional Encoding (P) tạo ra một vector vị trí độc lập, sau đó cộng trực tiếp vào Input Embedding (E). Positional encoding không thay thế embedding, nó dung hợp vị trí vào ngữ nghĩa.

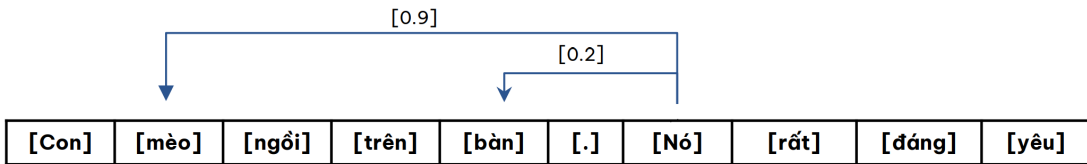
$$X = E + P$$

- **E**: Input Embeddings (Nội dung)
- **P**: Positional Encodings (Vị trí)
- **X**: Final Input (Kết quả)



Ngôn ngữ con người đầy sự mập mờ. Một từ đứng một mình thường không có ý nghĩa trọn vẹn. Khi mô hình AI xử lý một từ (token), nó không nhìn từ đó một cách cô lập.

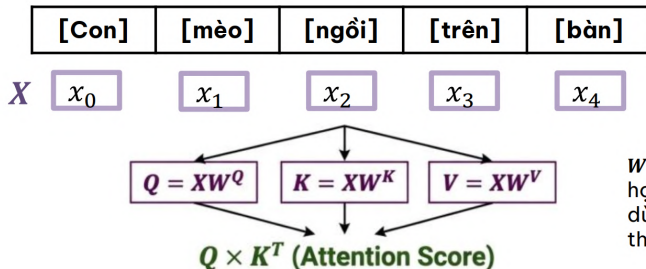
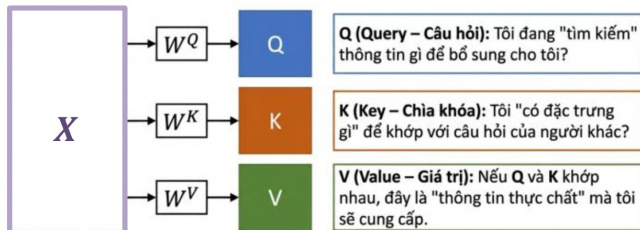
Attention — Cơ chế cho phép model “chú ý” đến các phần quan trọng nhất của input khi xử lý mỗi token.



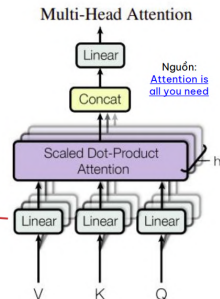
Ví dụ: “Con **mèo** ngồi trên bàn. **Nó** rất đáng yêu.”

Khi xử lý từ “Nó”, attention sẽ gán trọng số cao cho “mèo”, hiểu được “Nó” chỉ con mèo, không phải cái bàn.

Self-Attention giống như một hệ thống “khớp lệnh”. Lấy Query của token này đối chiếu với Key của mọi token khác để rút ra Value tương ứng.



W là ma trận trọng số học được của lớp linear dùng để biến input X thành Q, K, V .

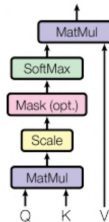


Tính “điểm giống nhau” (dot-product).
Điểm càng cao, 2 token càng liên quan.

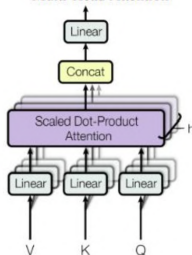
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Yếu tố scale (chia để làm
giảm độ lớn, d_k là số chiều).

Scaled Dot-Product Attention



Multi-Head Attention

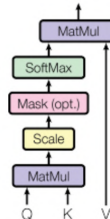


Nguồn: [Attention is all you need](#)

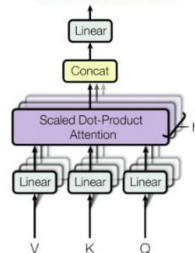
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Bước 1: Chấm điểm (Attention Scoring). Tính QK^T . Mỗi token “hỏi” mọi token khác, kể cả chính nó, để tạo ra ma trận điểm thô.

Scaled Dot-Product Attention



Multi-Head Attention



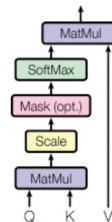
Nguồn: [Attention is all you need](#)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

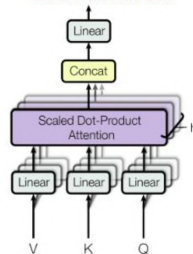
Bước 1: Chấm điểm (Attention Scoring). Tính QK^T . Mỗi token “hỏi” mọi token khác, kể cả chính nó, để tạo ra ma trận điểm thô.

Bước 2: Ổn định (Scaling). Chia cho $\sqrt{d_k}$ để làm dịu các điểm số quá lớn, tránh làm mô hình bị mất ổn định.

Scaled Dot-Product Attention



Multi-Head Attention

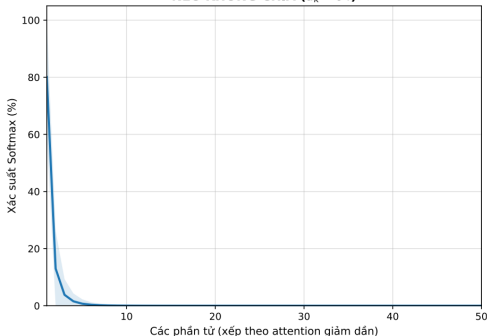


Nguồn: [Attention is all you need](#)

Bước 2: Ổn định (Scaling). Chia cho $\sqrt{d_k}$ để làm dịu các điểm số quá lớn, tránh làm mô hình bị mất ổn định.

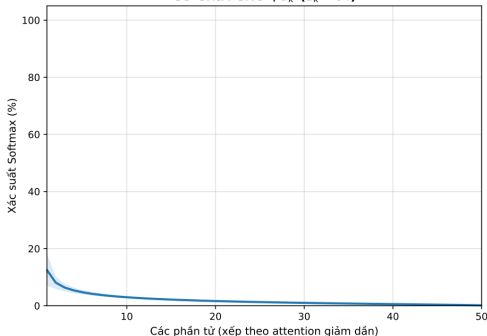
Tại sao phải chia cho $\sqrt{d_k}$? So sánh phân phối attention trước và sau scaling

NẾU KHÔNG CHIA ($d_k = 64$)



Tích vô hướng QK^T sinh ra các số khổng lồ. Hàm Softmax trở nên rất "nhọn" (gần với dạng one-hot). Mô hình ngừng học (Vanishing gradient).

CÓ CHIA CHO $\sqrt{d_k}$ ($d_k = 64$)



Kéo điểm số về mức nhỏ vừa phải, đồ thị Softmax phân phối "mềm mại" và dàn trải hơn. Gradients ổn định.

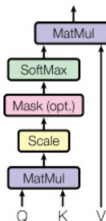
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Bước 1: Chấm điểm (Attention Scoring). Tính QK^T . Mỗi token “hỏi” mọi token khác, kể cả chính nó, để tạo ra ma trận điểm thô.

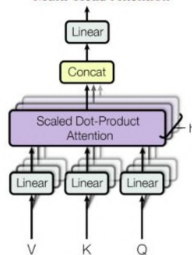
Bước 2: Ổn định (Scaling). Chia cho $\sqrt{d_k}$ để làm dịu các điểm số quá lớn, tránh làm mô hình bị mất ổn định.

Bước 3: Chuẩn hóa (Softmax). Áp dụng softmax để ép các điểm số thành trọng số trong khoảng từ 0 đến 1, sao cho tổng mỗi hàng bằng 1. Kết quả thu được attention map.

Scaled Dot-Product Attention



Multi-Head Attention



Nguồn: [Attention is all you need](#)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

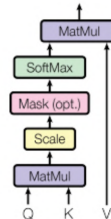
Bước 1: Chấm điểm (Attention Scoring). Tính QK^T . Mỗi token “hỏi” mọi token khác, kể cả chính nó, để tạo ra ma trận điểm thô.

Bước 2: Ổn định (Scaling). Chia cho $\sqrt{d_k}$ để làm dịu các điểm số quá lớn, tránh làm mô hình bị mất ổn định.

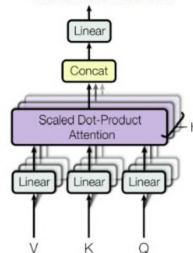
Bước 3: Chuẩn hóa (Softmax). Áp dụng softmax để ép các điểm số thành trọng số trong khoảng từ 0 đến 1, sao cho tổng mỗi hàng bằng 1. Kết quả thu được attention map.

Bước 4: Trộn thông tin (Weighted Sum). Nhân các trọng số chú ý với V để gom góp thông tin từ các token khác theo mức độ quan trọng đã tính ở Bước 3.

Scaled Dot-Product Attention

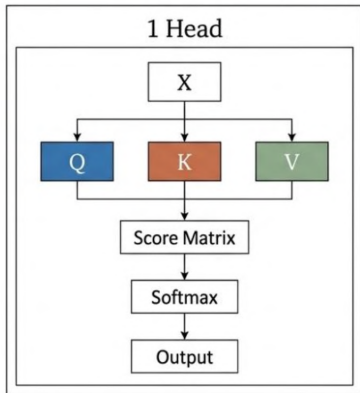


Multi-Head Attention



Nguồn: [Attention is all you need](#)

Khi áp dụng toàn bộ công thức attention chỉ với một bộ trọng số duy nhất, ta gọi đó là Single-Head Attention. Với chỉ một head, mô hình học cách hội tụ sự chú ý vào một kiểu quan hệ hay một khía cạnh ngữ cảnh cụ thể.

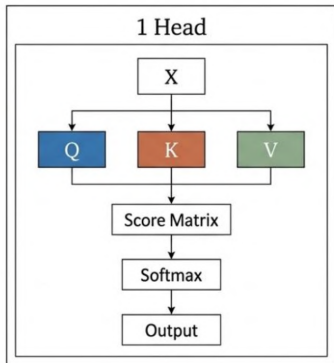


Ví dụ: Tính self-attention. Xét chuỗi 3 token:

[Tôi] [yêu] [AI]

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad d_k = 2$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Ví dụ:

Tính self-attention. Xét chuỗi 3 token: [Tôi] [yêu] [AI]

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, d_k = 2$$

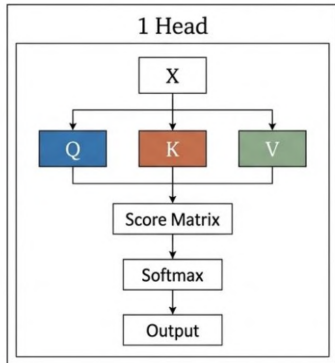
Bước 1: Chấm điểm (Attention Scoring): $QK^T = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

Bước 2: Ổn định (Scaling): $\frac{QK^T}{\sqrt{2}} \approx \begin{bmatrix} 0.7071 & 0.7071 & 0 \\ 0.7071 & 1.4142 & 0.7071 \\ 0 & 0.7071 & 0.7071 \end{bmatrix}$

Bước 3: Chuẩn hóa (Softmax):

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{bmatrix} 0.4011 & 0.4011 & 0.1978 \\ 0.2483 & 0.5035 & 0.2483 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix} \quad [\text{Tôi}]$$

Hàng 1 = token [Tôi] đang attend tới toàn bộ chuỗi



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Ví dụ:

Tính self-attention. Xét chuỗi 3 token: [Tôi] [yêu] [AI]

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, d_k = 2$$

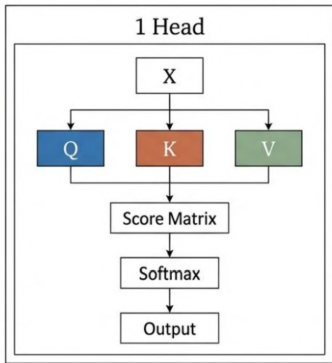
Bước 3: Chuẩn hóa (Softmax):

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{bmatrix} 0.4011 & 0.4011 & 0.1978 \\ 0.2483 & 0.5035 & 0.2483 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix} \quad [\text{Tôi}]$$

Hàng 1 = token [Tôi] đang attend tới toàn bộ chuỗi

- [Tôi] chú ý tới [Tôi] với trọng số 0.4011
- [Tôi] chú ý tới [yêu] với trọng số 0.4011
- [Tôi] chú ý tới [AI] với trọng số 0.1978

➔ [Tôi] lấy thông tin từ toàn câu, ưu tiên nhiều hơn cho [Tôi] và [yêu], ít hơn cho [AI]



Ví dụ:

Tính self-attention. Xét chuỗi 3 token: [Tôi] [yêu] [AI]

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, d_k = 2$$

Bước 3: Chuẩn hóa (Softmax):

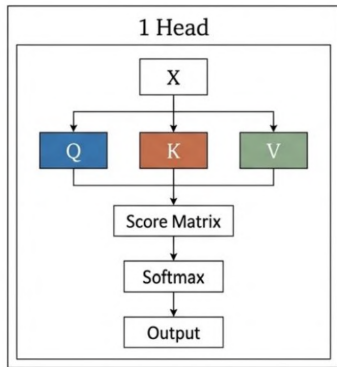
$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{2}} \right) \approx \begin{bmatrix} 0.4011 & 0.4011 & 0.1978 \\ 0.2483 & 0.5035 & 0.2483 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix} \text{ [yêu]}$$

Hàng 2 = token [yêu] đang attend tới toàn bộ chuỗi

- [yêu] chú ý tới [Tôi] với trọng số 0.2483
- [yêu] chú ý tới [yêu] với trọng số 0.5035
- [yêu] chú ý tới [AI] với trọng số 0.2483

→ [yêu] chú ý mạnh nhất vào chính nó, đồng thời vẫn lấy thông tin từ chủ ngữ [Tôi] và đối tượng [AI]

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Ví dụ:

Tính self-attention. Xét chuỗi 3 token: [Tôi] [yêu] [AI]

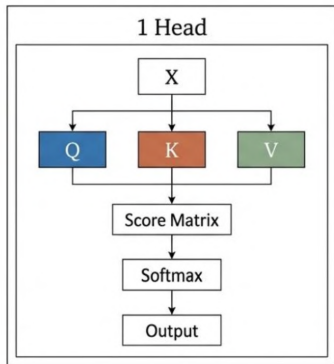
$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, d_k = 2$$

Bước 3: Chuẩn hóa (Softmax):

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{bmatrix} 0.4011 & 0.4011 & 0.1978 \\ 0.2483 & 0.5035 & 0.2483 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix} \begin{matrix} \\ \\ \text{[AI]} \end{matrix}$$

Tương tự, hàng 3?

- [AI] chú ý tới [Tôi] với trọng số 0.1978
 - [AI] chú ý tới [yêu] với trọng số 0.4011
 - [AI] chú ý tới [AI] với trọng số 0.4011
- ➔ [AI] lấy khá nhiều thông tin từ [yêu] và chính nó, ít hơn từ [Tôi]



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Ví dụ:

Tính self-attention. Xét chuỗi 3 token: [Tôi] [yêu] [AI]

$$Q = K = V = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, d_k = 2$$

Bước 1: Chấm điểm (Attention Scoring): $QK^T = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 1 \end{bmatrix}$

Bước 2: Ổn định (Scaling): $\frac{QK^T}{\sqrt{2}} \approx \begin{bmatrix} 0.7071 & 0.7071 & 0 \\ 0.7071 & 1.4142 & 0.7071 \\ 0 & 0.7071 & 0.7071 \end{bmatrix}$

Bước 3: Chuẩn hóa (Softmax):

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) \approx \begin{bmatrix} 0.4011 & 0.4011 & 0.1978 \\ 0.2483 & 0.5035 & 0.2483 \\ 0.1978 & 0.4011 & 0.4011 \end{bmatrix}$$

Bước 4: Trộn thông tin (Weighted Sum):

$$O = AV \approx \begin{bmatrix} 0.8022 & 0.5989 \\ 0.7517 & 0.7517 \\ 0.5989 & 0.8022 \end{bmatrix}$$

Giả sử sau khi tính điểm attention, ta có ma trận score:

$$S = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 2 & 4 \\ 0 & 1 & 2 \end{bmatrix}$$

Nếu **không mask**, token ở hàng 1 có thể nhìn cả 3 cột, kể cả tương lai. Điều này không được phép trong decoder-only.

Cơ chế:

- Thêm Ma trận Mask (M) chứa giá trị $-\infty$ vào trước hàm Softmax.
- Khi $\text{Softmax}(-\infty)$, tỷ lệ chú ý bị ép bằng 0%.

Dùng causal mask:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Khi đó:

$$S' = S + M = \begin{bmatrix} 2 & -\infty & -\infty \\ 1 & 2 & -\infty \\ 0 & 1 & 2 \end{bmatrix}$$

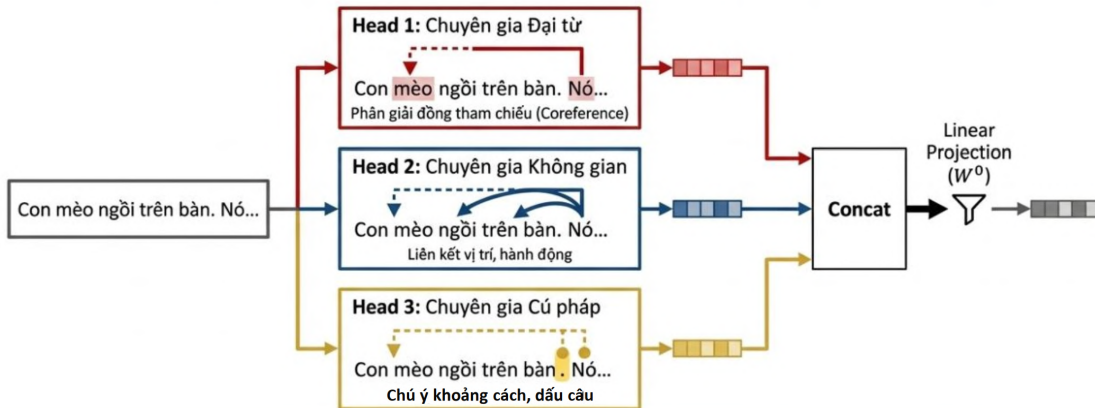
Như vậy: $A = \text{softmax}(S') \approx \begin{bmatrix} 1 & 0 & 0 \\ 0.2689 & 0.7311 & 0 \\ 0.0900 & 0.2447 & 0.6652 \end{bmatrix}$

Khi dạy AI tự động sinh văn bản (Decoder-only), ta phải che (**mask**) các từ chưa được sinh ra để chống gian lận. Mask biến Self-Attention từ dạng phân tích tĩnh thành một cỗ máy dự đoán tương lai tuyến tính hoàn hảo.



Token [Tôi] -> Chỉ được nhìn [Tôi].
Token [yêu] -> Được nhìn [Tôi], [yêu].
Token [AI] -> Được nhìn toàn bộ.

Multi-Head Attention - Thay vì chỉ học một kiểu quan hệ duy nhất, Multi-Head Attention tách attention thành nhiều head song song. Mỗi head học một góc nhìn khác nhau của câu. Các kết quả này sau đó được **concat** và đi qua **linear projection** để tạo ra biểu diễn cuối cùng giàu thông tin hơn.



Token — Đơn vị nhỏ nhất mà LLM xử lý — khoảng 0.75 từ tiếng Anh, 0.5 từ tiếng Việt

Tokenization: Tách text thành subword units

"Hello world" → 2 tokens

"Xin chào" → 3–4 tokens

"anthropic" → 3 tokens

"def func():" → 4 tokens

[``Tôi'' ] [ ``yêu'']

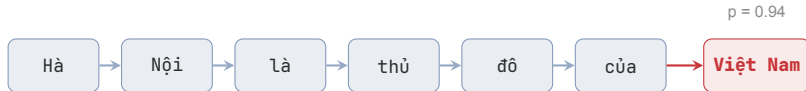
[``Việt'' ] [ ``Nam'']

→ 4+ tokens (mỗi từ có dấu = 1–2 tokens)

So sánh: "I love Vietnam" → 3 tokens

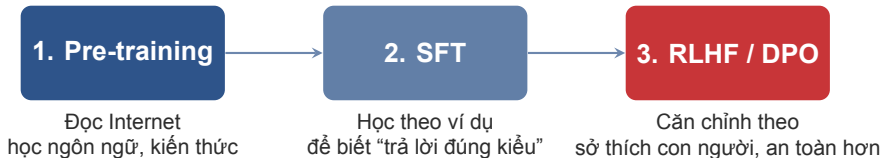
Thử: platform.openai.com/tokenizer

Lưu ý: Tiếng Việt tốn nhiều token hơn tiếng Anh (dấu, ký tự Unicode)
→ chi phí API cao hơn cho cùng nội dung.



- LLM **không “hiểu”** ngôn ngữ — nó dự đoán token có xác suất cao nhất
- **Temperature**: Điều chỉnh độ “sáng tạo” (0 = deterministic, 1 = random hơn)
- **Autoregressive**: Output token trở thành input cho bước tiếp theo

Lưu ý: LLM có thể tự tin đưa ra thông tin sai (hallucination) vì nó tối ưu xác suất, không phải sự thật.



Lưu ý: Analogy dễ nhớ: Pre-training = “đọc rất nhiều”, SFT = “được chỉ cách trả lời”, RLHF/DPO = “được uốn nắn để cư xử đúng hơn”.

Knowledge cutoff

Model không biết những gì xảy ra sau thời điểm training nếu không được cấp thêm dữ liệu/tools.

Hallucination

Model có thể trả lời rất tự tin nhưng sai vì đang tối ưu xác suất token, không phải tính đúng-sai.

Context window

Model chỉ “nhìn” được lượng token hữu hạn trong mỗi lần gọi. Quá dài thì tốn chi phí, và thông tin giữa prompt dễ bị quên.

Analogy

LLM giống một “học giả đọc rất nhiều” nhưng sống trong một bong bóng thời gian và chỉ được nhìn một số trang trước mặt.

Gợi ý dạy: nhấn mạnh rằng các giới hạn này giải thích vì sao sau này cần prompt tốt, context management, RAG và tools.

Token Economy

Chi phí, tốc độ và cách tính giá API

03

Token = đơn vị nhỏ nhất mà LLM xử lý

Ví dụ Tokenization

- "Hello world" → 2 tokens
- "Xin chào" → 3–4 tokens
- "def func():" → 4 tokens

LLM không đọc "từ",
LLM đọc **subword tokens**

Token được dùng để

- Tính chi phí API
- Giới hạn context window
- Đo độ dài prompt
- Quyết định latency

Công thức chi phí

Input tokens + Output tokens = Cost

Các trường hợp phổ biến

- **Tiếng Việt** — Unicode và từ bị tách nhỏ hơn "Tôi yêu Việt Nam" > "I love Vietnam"
- **Code** — nhiều ký tự đặc biệt và khoảng trắng `def func():` → nhiều tokens
- **Text có cấu trúc** — JSON, URL, ID, số dài `user_id: 98347298347`

Rule of Thumb

Unicode + ký tự đặc biệt + cấu trúc phức tạp → tốn nhiều token hơn



- Giá tính theo **1 triệu tokens** (1M tokens)
- Output tokens **đắt hơn** input tokens (3–5x)
- Giá giảm $\sim 10x$ mỗi năm (GPT-4 level: \$20/M $\rightarrow$ \$2/M trong 2 năm)

Nguồn làm tăng chi phí

- Input tokens chiếm phần lớn chi phí
- System prompt lặp lại mỗi API call
- RAG context dài → cost cao
- Chat history dài → cost tăng dần

Ví dụ

User question: 50 tokens
System prompt: 300 tokens
RAG context: 800 tokens
Output: 200 tokens
Total = 1350 tokens / call

Kết luận

Tối ưu chi phí = tối ưu prompt + context

Tăng Latency

- Context dài hơn
- Output dài hơn
- Model lớn hơn

Tăng Cost

- Nhiều input tokens
- Nhiều output tokens
- Model đắt hơn

Key Insight

Nhiều tokens hơn → vừa chậm hơn vừa đắt hơn

Model	In	Out	Ctx	Loại	Khi nên dùng
Claude Opus 4.6	\$5.0	\$25	1M	Closed	Reasoning, code khó
Claude Sonnet 4	\$3.0	\$15	1M	Closed	Balanced choice
Claude Haiku 4.5	\$0.8	\$4	200K	Closed	Fast, cheap, routing
GPT-4o	\$5.0	\$20	128K	Closed	Multimodal, ecosystem
Gemini 2.5 Pro	\$1.25	\$10	1M	Closed	Long-context tasks
Llama 4 Scout	Free	Free	1M	Open	Self-host, private data

Lưu ý: Closed = API hosted; Open = self-host / control nhiều hơn. Giá tham khảo tháng 3/2026.

Nếu ưu tiên cost/latency

- FAQ, phân loại, trích xuất đơn giản
- Batch jobs số lượng lớn
- Trả lời ngắn, ít reasoning

Gợi ý: Haiku, Gemini Flash, model nhỏ/open-source

Nếu ưu tiên quality/reasoning

- Phân tích nhiều bước, code, planning
- Tài liệu dài, ngữ cảnh phức tạp
- Bài toán cần độ tin cậy cao

Gợi ý: Sonnet, Opus, GPT-4o, Gemini Pro

Lưu ý: Rule of thumb: bắt đầu từ model **đủ tốt và đủ rẻ**. Chỉ nâng model khi chất lượng thực sự chặn use case.

Prompt ví dụ: “Tóm tắt báo cáo tài chính Q1 trong 3 bullet và nêu 1 rủi ro chính.”

Claude

Mạch lạc, thiên về cấu trúc.

Phong cách: cẩn thận, “consulting style”.

GPT-4o

Ngắn gọn, tự nhiên, linh hoạt.

Phong cách: hợp app/chat, đa dụng.

Gemini

Mạnh khi context dài, nhiều tài liệu.

Phong cách: hợp workflow nhiều file.

*Gợi ý dạy: chạy live cùng 1 prompt để học viên thấy model selection không chỉ là “giá”, mà còn là **phong cách + độ phù hợp task**.*

Context Window — Số token tối đa mà LLM xử lý trong 1 lần gọi API (input + output)

- **128K tokens** $\approx$ 1 cuốn sách 300 trang
- **1M tokens** $\approx$ 4–5 cuốn sách
- Context càng dài $\rightarrow$ chi phí càng cao
- Thông tin ở giữa context dễ bị “quên” (Lost in the Middle)

Gemini 2.5	1M
Claude Sonnet 4	1M
Claude Opus 4.6	1M
GPT-4o	128K
Llama 4	1M

Scenario: Chatbot hỗ trợ khách hàng, 1000 lượt/ngày

Input trung bình: 500 tokens/lượt (câu hỏi + context)

Output trung bình: 200 tokens/lượt (câu trả lời)

Dùng Claude Sonnet 4:

Input: $500K \times \$3/1M = \1.50

Output: $200K \times \$15/1M = \3.00

Tổng/ngày: \$4.50

Tổng/tháng: ~\$135

Dùng Claude Haiku 4.5:

Input: $500K \times \$0.80/1M = \0.40

Output: $200K \times \$4/1M = \0.80

Tổng/ngày: \$1.20

Tổng/tháng: ~\$36

Lưu ý: Chọn model phù hợp: Haiku cho tác vụ đơn giản, Sonnet/Opus cho reasoning phức tạp.

Gọi API lần đầu

Từ “Hello World” đến production-ready call

04



- **Prompt:** system + user input + context
- **API Call:** gửi request tới model provider
- **Token Stream:** model sinh output từng chunk
- **Response:** text hoàn chỉnh + usage + stop reason

Lưu ý: Tư duy đúng cho PM/engineer: mỗi API call luôn có 3 thứ cần kiểm soát cùng lúc: **quality, latency, cost.**

- ✓ Python 3.10+ đã cài đặt
- ✓ VS Code hoặc Cursor IDE
- ✓ Tài khoản Open API (có credit)
- ✓ Biến môi trường: OPENAI_API_KEY
- ✓ Tài khoản Google Colab

```
from dotenv import load_dotenv
import os
from openai import OpenAI

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY")
client = OpenAI(api_key=api_key)

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Hello!"}
    ]
)

print(response.choices[0].message.content)
```

Request (gửi đi):

- `model`: model sử dụng (vd: gpt-4o)
- `messages`: input hội thoại
- `max_tokens`: giới hạn độ dài output
- `temperature`: độ sáng tạo (tùy chọn)

Ví dụ:

- `role`: system / user / assistant
- `messages` = list hội thoại

Response (nhận về):

- `choices[0].message.content`: nội dung trả lời
- `model`: model thực tế dùng
- `usage.prompt_tokens`: input tokens
- `usage.completion_tokens`: output tokens
- `usage.total_tokens`: tổng tokens
- `finish_reason`: stop | length | tool_calls

temperature	Độ “sáng tạo” (0–1)	0 = deterministic; 1 = diverse. Dùng thấp cho code/phân tích, cao hơn cho sáng tạo
top_p	Nucleus sampling (0–1)	Chỉ chọn từ top tokens chiếm $p\%$ xác suất. Thường dùng 0.9–0.95
stop_sequences	Dừng ở chuỗi chỉ định	Hữu ích khi cần output có cấu trúc cố định hoặc cắt đúng điểm mong muốn

UNDERSTANDING TEMPERATURE IN LARGE LANGUAGE MODELS (LLMs)

DETERMINISTIC MODEL ($T=0$):
Always picks the word with the highest absolute probability.

INCREASED CREATIVITY ($0 < T < 2$):
Encourages novelty by allowing the model to consider words with lower initial probabilities.

STRONG RANDOM BIAS ($T \approx 2$):
Significantly shifts probabilities towards unusual or unexpected outputs.

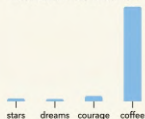
0 1.0 2.0
TEMPERATURE VALUE (T)

"A cup of coffee."

"A cup of courage."

"A cup of stars."

PROBABILITY DISTRIBUTION



PROBABILITY DISTRIBUTION



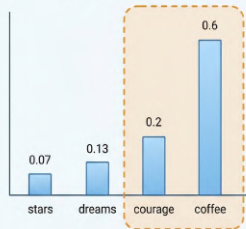
PROBABILITY DISTRIBUTION



Top-p Sampling

A visual guide to Nucleus Sampling

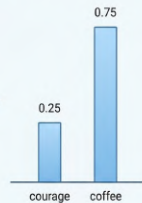
Original Distribution



Resampling

Normalize selected probabilities

New Distribution



Bắt đầu với temperature=0 cho tác vụ cần ổn định. Chỉ tăng sampling khi thật sự cần đa dạng câu trả lời.

```
# === OPENAI (GPT) ===
from openai import OpenAI
client = OpenAI()
resp = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Hello"}]
)
print(resp.choices[0].message.content) # .choices[0]...

# === ANTHROPIC (Claude) ===
import anthropic
client = anthropic.Anthropic()
resp = client.messages.create(
    model="claude-sonnet-4-6", max_tokens=1024,
    messages=[{"role": "user", "content": "Hello"}]
)
print(resp.content[0].text) # .content[0].text
```

```
from openai import OpenAI

client = OpenAI(
    base_url=f"<YOUR_API_ENDPOINT>",
    api_key='<YOUR_API_KEY>',
)

def call_llm(prompt):
    response = client.chat.completions.create(
        model="<MODEL_NAME>",
        messages=[
            {"role": "user", "content": prompt}
        ],
        max_tokens=300
    )

    return response.choices[0].message.content

print(call_llm("Explain RAG in 2 bullets"))
```

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Explain tokenization"}
    ],
    max_tokens=200
)

print(response.choices[0].message.content)

print("Prompt tokens:", response.usage.prompt_tokens)
print("Completion tokens:", response.usage.completion_tokens)
print("Total tokens:", response.usage.total_tokens)
print("Finish reason:", response.choices[0].finish_reason)
```

```
from openai import OpenAI

client = OpenAI()

while True:
    user_input = input("You: ")

    if user_input.lower() in ["exit", "quit"]:
        break

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[
            {"role": "user", "content": user_input}
        ],
        max_tokens=300
    )

    print("Bot:", response.choices[0].message.content)
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen3-0.6B-Base"

# Load the tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load the model
model = AutoModelForCausalLM.from_pretrained(model_name)

# Example of generating text (optional)
inputs = tokenizer("Hello, world!", return_tensors="pt")
outputs = model.generate(inputs["input_ids"], max_new_tokens=100)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

```
from openai import OpenAI

client = OpenAI()

# Streaming: receive response chunk by chunk
stream = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "user", "content": "Write a poem"}
    ],
    stream=True,
    max_tokens=1024
)

for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content,
              end="", flush=True)
```


Vibe Coding

Lập trình bằng cách làm việc cùng AI

05

Định nghĩa

Viết phần mềm bằng cách mô tả ý tưởng AI sẽ generate code

- Không viết code từ đầu
- Mô tả yêu cầu bằng ngôn ngữ tự nhiên
- AI sinh code
- Developer review và chỉnh sửa

- Viết code thủ công chậm
- Boilerplate code lặp lại
- AI viết code nhanh hơn
- Tập trung vào logic thay vì syntax

Viết phần mềm nhanh hơn không phải viết code nhiều hơn

Quy trình Vibe Coding

Idea → Prompt → Code → Test → Refine

- Mô tả bài toán
- AI generate code
- Chạy thử nhanh
- Refine prompt
- Lặp lại nhiều lần
- Chốt kết quả

Cách lập trình truyền thống

- Nghĩ thuật toán trước
- Viết code từng bước
- Debug lỗi thủ công
- Tối ưu hiệu năng
- Boilerplate nhiều

Vibe Coding Mindset

- Mô tả mục tiêu rõ ràng
- AI generate code
- Chỉnh sửa bằng prompt
- Review logic quan trọng
- Iterate nhanh nhiều lần

Mindset mới

Developer chuyển từ **người viết code** → sang **người thiết kế + review + điều phối AI**

1. Intent-driven

Nói rõ mục tiêu
và output mong muốn

2. Context-first

Cung cấp bối cảnh
file, ví dụ, ràng buộc

3. Human review

AI viết nhanh
con người kiểm tra và chốt

Vibecoding hiệu quả khi ý định rõ ràng, ngữ cảnh đầy đủ và luôn có bước rà soát cuối.

Prompt kém

Write a chatbot using OpenAI

- Mục tiêu mơ hồ
- Thiếu yêu cầu cụ thể
- Không có tiêu chí đầu ra
- AI dễ trả code chung chung

Prompt tốt

Build a CLI chatbot using OpenAI

- conversation memory
- streaming response
- exit with “quit”
- show token usage

Kết quả: output rõ hơn, dễ dùng hơn, ít phải sửa hơn

Thực hành

Live Demo & Lab

06

Mục tiêu: Gọi OpenAI API thực tế: so sánh GPT-4o và GPT-4o-mini về latency, cost, quality

Deliverable: Script Python hoàn chỉnh: gọi GPT-4o + GPT-4o-mini, chatbot có streaming, bảng so sánh kết quả

Thời gian: 90 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 LLM = Transformer dự đoán token tiếp theo từ context
- 2 Để usable, LLM đi qua Pre-training → SFT → alignment
- 3 Chọn model theo trade-off quality, latency, cost
- 4 Một API call luôn có prompt, response, usage và stop reason
- 5 Vibe Coding tốt = intent rõ + context đủ + review kỹ

Ngày 2: Prompt Engineering

“Prompt tốt tạo ra output tốt. Nhưng “tốt” nghĩa là gì?”

- Đọc: “Attention Is All You Need” (2017)
- Thử gọi API với 3 prompts khác nhau

1. Vaswani et al. (2017). “Attention Is All You Need”. arXiv:1706.03762
2. Ouyang et al. (2022). “InstructGPT / RLHF”. arXiv:2203.02155
3. Rafailov et al. (2023). “DPO”. arXiv:2305.18290
4. Karpathy (2023). “State of GPT” practitioner talk
5. Anthropic / OpenAI / Google API quickstarts

Hỏi & Đáp

Bạn có câu hỏi nào về LLM, Transformer, Token Economy, hoặc API?



Cảm ơn!

Huỳnh Thành Trung

Email: trung.ht@vinuni.edu.vn

Bài tập nộp trước Ngày 2 · Đọc thêm: Vaswani et al. (2017)